

DOCUMENTAÇÃO DO PIPELINE DE DADOS

Projeto Eco360 — Raiz Inteligente | Do CSV bruto ao modelo relacional em MySQL, passo a passo

Este documento existe para você conseguir explicar depois, etapa por etapa, o que foi feito com os dados — desde os arquivos brutos recebidos até as tabelas normalizadas prontas para SQL e Power BI. Cada seção corresponde a uma decisão técnica tomada e o porquê dela.

1. Visão Geral do Pipeline

O pipeline segue 4 camadas, na ordem em que foram construídas:

Camada	O que contém	Formato
1. RAW	Arquivos exatamente como recebidos (PAM, INMET, MapBiomias, Emissões, base estadual)	CSV, separadores/decimais variados
2. Limpeza	Mesmos dados, mas com nomes de coluna padronizados, decimais corrigidos, formato longo (não largo)	CSV consistente
3. Modelo normalizado	dim_municipio + tabelas fato (soja, clima, cobertura, emissão)	CSV pronto para SQL
4. Carga	Tabelas dentro do MySQL (banco eco_360), via script Python	Banco relacional

Por que essa separação em camadas? Porque se algo parecer errado depois (ex: um número estranho no Power BI), dá pra rastrear em qual camada o problema entrou — no dado bruto, na limpeza, ou na modelagem. Misturar tudo em um único CSV gigante torna isso impossível de auditar depois.

2. Etapa 1 — Bases Brutas Recebidas

Cinco arquivos-fonte, cada um com sua própria origem e formato:

Arquivo original	Fonte	Problema de formato encontrado
soja_emissoes_2019_2024...csv	Emissões municipais (SEEG-like)	Anos como colunas (2019...2024) em vez de linhas; sem código IBGE
pam_mapbiomas_solo_integrado...csv	IBGE/PAM + MapBiomias Solo	Nenhum problema estrutural — já vinha com codigo_ibge
inmet_municipio_ano...csv	INMET (clima)	Coluna regioao em minúsculo/underscore (centro_oeste) diferente das outras bases (Centro-Oeste)
mapbiomas_cobertura_municipio...csv	MapBiomias Cobertura	Nenhum problema estrutural
base_soja_centro_oeste_sul / base_tratada / taxa_emissao / area_conversao / emissao_absoluta	Base estadual (7 UFs)	Separador ';' + decimal ',' (padrão BR) misturado com arquivos usando ';' + '.'; nomes de coluna em maiúsculo (Estado, UF)

3. Etapa 2 — Limpeza e Padronização

3.1 Padronização de nomes de coluna

Todas as colunas foram convertidas para minúsculo, sem espaço e sem acento no nome (ex: Estado → estado, UF → uf). Isso é obrigatório para SQL — nomes de coluna com maiúscula/espaço geram erro ou exigem aspas em toda query.

3.2 Correção de separador decimal

Os arquivos da base estadual usavam vírgula como decimal e ponto como milhar (ex: 1.046.479,65) — padrão brasileiro. Isso quebra a leitura em ferramentas que esperam CSV internacional (ponto decimal, vírgula separando colunas). A correção aplicada:

```
valor_limpo = valor_texto.replace('.', '').replace(',', '.')
# "1.046.479,65" -> "1046479.65" -> convertido para float
```

3.3 Transformação de formato largo para longo (a mais importante)

A base de emissões trazia os anos como colunas:

municipio	uf	2019	2020	...	2024
Brasília	DF	20010.54	22276.10	...	22287.60

Isso impede qualquer agregação por ano em SQL sem escrever uma query gigante somando coluna por coluna. Foi convertido para formato longo (tidy), usando um melt/UNPIVOT:

codigo_ibge	municipio	uf	ano	emissao_t
5300108	Brasília	DF	2019	20010.54
5300108	Brasília	DF	2020	22276.10
...				

Com isso, uma pergunta como "emissão total por ano" vira uma única query:

```
SELECT ano, SUM(emissao_t)
FROM fato_emissao_soja
GROUP BY ano
ORDER BY ano;
```

3.4 Separação de dimensões misturadas em uma coluna só

A coluna gas trazia texto composto, misturando duas informações diferentes: "CO2e (t) GTP-AR2". Foi separada em duas colunas atômicas — metrica (GTP/GWP/N2O) e unidade (tCO2e/t) — porque uma coluna deve representar uma informação, não duas concatenadas. Isso também facilita filtro (WHERE metrica = 'GWP') sem precisar de LIKE '%GWP%'.

4. Etapa 3 — Modelagem Relacional (Normalização)

Em vez de um único CSV gigante repetindo município, UF e região em toda linha, o modelo foi dividido em uma dimensão (dim_municipio) e quatro tabelas fato — o padrão de modelagem dimensional usado em SQL para dados analíticos (schema estrela).

4.1 Por que separar em dimensão + fato?

- Sem repetição de dado: a área territorial de um município (area_km2) é sempre a mesma, independente do ano — guardar isso repetido em cada uma das 6 linhas anuais é redundante e gera risco de inconsistência (e se alguém editar 1 das 6 linhas e esquecer as outras?).
- Chave única e confiável: nome de município não é uma chave segura (existem municípios com nomes iguais em estados diferentes). codigo_ibge é único e oficial.
- Facilidade de JOIN: qualquer tabela fato se conecta a dim_municipio pela mesma chave (codigo_ibge), então todo relacionamento no Power BI ou em SQL segue o mesmo padrão.

4.2 Como dim_municipio foi construída

Foi feita a união (UNION) dos municípios presentes em três fontes — PAM (1.505 municípios), INMET (1.661) e MapBiomias (1.660) — para não perder nenhum município que aparecesse em pelo menos uma das bases:

```
SELECT DISTINCT codigo_ibge, municipio, uf, regiao FROM pam
UNION
SELECT DISTINCT codigo_ibge, municipio, uf, regiao FROM inmet
UNION
SELECT DISTINCT codigo_ibge, municipio, uf, regiao FROM mapbiomas;
-- resultado: 1.661 municípios únicos (o teto = INMET, que tinha a lista mais completa)
```

A coluna area_km2 foi obtida da MapBiomias (area_ibge_ha / 100) em vez da PAM, porque a MapBiomias cobria mais municípios (1.660 vs. 1.505) — validei antes que os dois valores batiam exatamente para os municípios em comum (diferença = 0 em 8.674 comparações), então usar a fonte com mais cobertura não introduziu inconsistência.

4.3 Como cada tabela fato foi construída

Tabela fato	Construção	O que foi removido e por quê
fato_soja	PAM/MapBiomias Solo direto, chave codigo_ibge+ano	municipio/uf/regiao/area_km2 (foram para dim_municipio); cultura (sempre "soja" nesta base — sem valor informativo)
fato_clima	INMET direto, chave codigo_ibge+ano	municipio/uf/regiao (foram para dim_municipio)
fato_cobertura	MapBiomias Cobertura direto, chave codigo_ibge+ano	municipio/uf/regiao (foram para dim_municipio)
fato_emissao_soja	Emissões após o melt (largo→longo) + JOIN com dim_municipio para trazer codigo_ibge (a base original só tinha município+UF, sem código)	colunas 100% constantes em toda a base (setor_de_emissao, categoria_emissora, produto_ou_sistema etc. — todas com um único valor em 34.524 linhas, sem poder de diferenciação)
fato_emissao_estado	Base estadual, após correção de decimal BR → float	Nada removido — granularidade (UF) é diferente das demais, por isso ficou como tabela separada, não junta com as outras

4.4 Enriquecimento com latitude/longitude (para mapas no Power BI)

Foi cogitado usar o arquivo INMET_completo.csv (dados diários por estação) como fonte de coordenadas, já que ele traz latitude/longitude. Isso foi descartado depois de verificar dois problemas:

- Cobertura baixa: o arquivo tem apenas 209 estações meteorológicas únicas, contra 1.661 municípios no modelo — cobriria só ~12,6% do mapa.

- Nome de estação ≠ nome de município: nomes como "GAMA (PONTE ALTA)" são bairros/localidades, não municípios. Um teste de correspondência direta (nome normalizado + UF) bateu em apenas 192 das 209 estações.

A solução usada foi buscar a tabela oficial de centróides do IBGE (codigo_ibge, latitude, longitude para os 5.570 municípios do Brasil) e cruzar com dim_municipio pela chave codigo_ibge:

```
SELECT d.*, g.latitude, g.longitude
FROM dim_municipio d
LEFT JOIN ibge_centroides g ON g.codigo_ibge = d.codigo_ibge;
-- resultado: 1.659 de 1.661 municípios com coordenada (99,9%)
```

Os 2 municípios sem coordenada ("Área Operacional Lagoa Mirim" e "Área Operacional Lagoa dos Patos", ambos no RS) não são municípios reais — são áreas de corpo d'água que entraram na base pela classificação do MapBiomas/INMET. Ficaram como NULL propositalmente, em vez de receber uma coordenada inventada.

As colunas foram adicionadas diretamente em dim_municipio (não em tabela separada), porque têm a mesma granularidade — 1 linha por codigo_ibge — e todo visual de mapa no Power BI já herda essas colunas automaticamente via o relacionamento existente com as tabelas fato.

5. Etapa 4 — Estrutura SQL (DDL)

Abaixo o CREATE TABLE de cada tabela, já com chave primária. As tabelas fato referenciam dim_municipio(codigo_ibge) como chave estrangeira.

5.1 dim_municipio

```
CREATE TABLE dim_municipio (
  codigo_ibge INT PRIMARY KEY,
  municipio VARCHAR(100) NOT NULL,
  uf CHAR(2) NOT NULL,
  regioao VARCHAR(20) NOT NULL,
  area_km2 DECIMAL(10,3),
  latitude DECIMAL(9,6),
  longitude DECIMAL(9,6)
);
```

5.2 fato_soja

```
CREATE TABLE fato_soja (
  codigo_ibge INT NOT NULL,
  ano SMALLINT NOT NULL,
  area_plantada_ha DECIMAL(12,2),
  area_colhida_ha DECIMAL(12,2),
  area_nao_colhida_ha DECIMAL(12,2),
  aproveitamento_area_pct DECIMAL(6,2),
  quantidade_produzida_t DECIMAL(14,2),
  rendimento_medio_kg_ha DECIMAL(10,2),
  valor_producao_mil_reais DECIMAL(14,2),
  carbono_solo_t_ha DECIMAL(8,2),
  tem_dado_producao TINYINT(1) NOT NULL DEFAULT 1,
  PRIMARY KEY (codigo_ibge, ano),
  FOREIGN KEY (codigo_ibge) REFERENCES dim_municipio(codigo_ibge)
);
```

5.3 fato_clima (colunas principais — as de origem/qualidade seguem o mesmo padrão repetido para temperatura e umidade)

```
CREATE TABLE fato_clima (  
  codigo_ibge          INT NOT NULL,  
  ano                  SMALLINT NOT NULL,  
  precipitacao_anual_mm  DECIMAL(8,2),  
  origem_precipitacao   VARCHAR(30),  
  numero_estacoes_observadas_precipitacao SMALLINT,  
  numero_estacoes_idw_precipitacao      SMALLINT,  
  temperatura_media_anual_c  DECIMAL(5,2),  
  origem_temperatura         VARCHAR(30),  
  umidade_media_anual_pct    DECIMAL(5,2),  
  origem_umidade            VARCHAR(30),  
  score_qualidade_climatica  TINYINT,  
  qualidade_climatica_geral  VARCHAR(20),  
  tipo_representacao_climatica VARCHAR(30),  
  -- (demais colunas de auditoria: qualidade_espacial_*, cobertura_observada_*_pct,  
  -- distancia_max_idw_*_km – mesmo padrão para precipitação/temperatura/umidade)  
  PRIMARY KEY (codigo_ibge, ano),  
  FOREIGN KEY (codigo_ibge) REFERENCES dim_municipio(codigo_ibge)  
);
```

5.4 fato_cobertura (colunas principais)

```
CREATE TABLE fato_cobertura (  
  codigo_ibge          INT NOT NULL,  
  ano                  SMALLINT NOT NULL,  
  area_agricultura_ha  DECIMAL(12,2),  
  area_soja_mapbiomas_ha DECIMAL(12,2),  
  area_cobertura_natural_ha DECIMAL(12,2),  
  area_pastagem_ha     DECIMAL(12,2),  
  pct_agricultura       DECIMAL(6,3),  
  pct_soja_mapbiomas    DECIMAL(6,3),  
  pct_cobertura_natural DECIMAL(6,3),  
  pct_pastagem          DECIMAL(6,3),  
  biomas_presentes      VARCHAR(100),  
  quantidade_biomas     TINYINT,  
  -- (demais colunas de área/percentual por classe de uso do solo)  
  PRIMARY KEY (codigo_ibge, ano),  
  FOREIGN KEY (codigo_ibge) REFERENCES dim_municipio(codigo_ibge)  
);
```

5.5 fato_emissao_soja

```
CREATE TABLE fato_emissao_soja (  
  id_emissao    INT PRIMARY KEY AUTO_INCREMENT,  
  codigo_ibge   INT,  
  ano           SMALLINT NOT NULL,  
  metodologia   VARCHAR(10) NOT NULL,  -- AR2, AR4, AR5, AR6, GASES  
  metrica       VARCHAR(10) NOT NULL,  -- GTP, GWP, N2O  
  unidade       VARCHAR(10) NOT NULL,  -- tCO2e, t  
  tipo_emissao  VARCHAR(20) NOT NULL,  -- Diretas, Indiretas  
  bioma         VARCHAR(30),  
  emissao_t     DECIMAL(16,2),  
  FOREIGN KEY (codigo_ibge) REFERENCES dim_municipio(codigo_ibge)  
);  
-- codigo_ibge aceita NULL: municípios da base de emissão que não  
-- têm correspondência confirmada em dim_municipio ficam com FK vazia
```

```
-- em vez de serem descartados (ver seção 6.3).
```

5.6 fato_emissao_estado

```
CREATE TABLE fato_emissao_estado (  
    uf                CHAR(2) PRIMARY KEY,  
    estado            VARCHAR(30) NOT NULL,  
    regioao           VARCHAR(20) NOT NULL,  
    cultura            VARCHAR(20) NOT NULL,  
    taxa_emissao       DECIMAL(8,2),  
    taxa_ic_inferior    DECIMAL(8,2),  
    taxa_ic_superior    DECIMAL(8,2),  
    emissao_absoluta    DECIMAL(16,2),  
    emissao_ic_inferior DECIMAL(16,2),  
    emissao_ic_superior DECIMAL(16,2),  
    area_conversao      DECIMAL(14,2)  
);  
-- Não tem FK para dim_municipio: a granularidade é estado (uf),  
-- não município. Fica como tabela de contexto complementar.
```

6. Etapa 5 — Script de Carga (ETL Python → MySQL)

O script `etl_eco360.py` automatiza a carga de todas as tabelas no banco `eco_360`:

1. Baixa (ou lê localmente) os 6 arquivos CSV já tratados da pasta `./base`.
2. Conecta no MySQL e cria o banco `eco_360` se ainda não existir.
3. Lê cada CSV com fallback automático de separador (tenta vírgula; se resultar em 1 coluna só, tenta ponto e vírgula) e de encoding (`utf-8-sig`; se falhar, `latin1`) — proteção contra o problema de formato BR vs. internacional descrito na seção 3.2.
4. Carrega cada DataFrame como tabela no MySQL (`to_sql` com `if_exists='replace'`), na ordem: `dim_municipio` primeiro, depois as tabelas `fato` — porque elas dependem da dimensão já existir para a FK funcionar.
5. Aplica a chave primária de cada tabela após a carga, já que `to_sql` não define PK/FK automaticamente.

6.1 Trecho principal (mapeamento tabela → arquivo)

```
arquivos = {  
    'dim_municipio': 'dim_municipio.csv',  
    'fato_soja': 'fato_soja.csv',  
    'fato_clima': 'fato_clima.csv',  
    'fato_cobertura': 'fato_cobertura.csv',  
    'fato_emissao_soja': 'fato_emissao_soja.csv',  
    'fato_emissao_estado': 'fato_emissao_estado.csv',  
}
```

7. Etapa 6 — Achados de Qualidade de Dados

Três pontos foram verificados e documentados durante o processo, para não serem confundidos com erro do pipeline mais tarde:

7.1 Nulos no clima não são erro — são metodológicos

Em fato_clima, colunas como numero_estacoes_observadas_* e numero_estacoes_idw_* nunca aparecem preenchidas ao mesmo tempo na mesma linha: cada linha usa OU dado observado direto de estação INMET (684 linhas) OU dado interpolado por IDW (9.282 linhas), nunca os dois. Preencher esses nulos com 0 faria parecer "zero estações" quando na verdade a métrica nem se aplica àquele método.

7.2 Cinco municípios com produção ausente na PAM

Porto Rico (PR, 2024), Canoas (RS, 2024), Cidreira (RS, 2024), Imbé (RS, 2024) e Parobé (RS, 2020) têm valores ausentes em area_colhida_ha, quantidade_produzida_t e colunas relacionadas — apesar do campo status_dado dizer "disponível" (não é confiável como flag). Por isso foi criada a coluna tem_dado_producao (0/1) em vez de usar status_dado ou preencher com zero.

7.3 "Emissão por tonelada" é praticamente um valor fixo

Ao testar emissao_t / quantidade_produzida_t como indicador de eficiência ambiental por município (para o Índice de Oportunidade e a página de Potencial de Investimento), o valor ficou em ≈0,095 tCO2e/t em 99,6% dos 8.669 registros — ou seja, é um fator de emissão fixo (padrão IPCC) aplicado uniformemente pela metodologia da base, não uma medida real de eficiência de manejo por município. Documentado para não ser usado como diferenciador em análises futuras.

8. Onde Isso Está Documentado no Restante do Projeto

Arquivo	O que contém
planejamento_power_bi_eco360.docx	Colunas por página do dashboard, gráficos recomendados, DAX do IOA, monitoramento e potencial de investimento
eco360_mysql.sql	DDL completo (CREATE TABLE + FOREIGN KEY + índices) e comandos LOAD DATA LOCAL INFILE para carga direta via cliente MySQL
etl_eco360.py	Script Python alternativo de carga (download do Drive + pandas.to_sql), sem precisar alterar permissão local_infile do servidor
documentacao_pipeline_eco360.docx (este arquivo)	Como os dados brutos viraram o modelo relacional — a explicação de cada decisão tomada

Os quatro documentos se complementam: este explica "como os dados chegaram até aqui"; o de planejamento explica "o que fazer com eles no Power BI"; e os dois scripts (.sql e .py) são as duas formas alternativas de efetivamente carregar tudo no MySQL.